UTORONTO
EFA

Description of Problem

For many years, our team has used a barely changed, in-house C++ program, PANDA, created in 2008 to simulate large numbers of canoe designs, and a Python script, POSSUM, created in 2019 to converge on optimal designs according to user defined criteria. Given new regulations and races introduced in recent years, the team decided that revising the code was essential to ensure its calculations and scoring more accurately represent expected racing conditions.

Background Research, Technical Approach, and Assumptions

The team spent the first half of the year studying the old code and discussing the changes that would be the most impactful. Testing and reviewing the old C++ code revealed the need to add various features and improve input flexibility, which was the initial scope of the team's work. However, it was also found that the code had to be rewritten because the original C++ script was using outdated libraries and functions, so could not compile as is. Thus, it was also determined that the team would rewrite the script in Python, and consolidate it with the secondary python script. Using Python specifically would improve readability and since it is one of the most commonly used languages today [22], accordingly it will be easier for present and future team members to understand. In Python, we could also leverage certain data libraries like Pandas and NumPy, allowing for easy data storage and processing.

To better simulate all racing conditions, the inputs were adjusted to accept any number of loadcases, with any number of paddlers for whom specific weights, heights and genders would be considered in calculations. Using gender and height information, a more accurate vertical center of mass is now being calculated using equations from research studies [23]. There is also an option to consider backwards paddling as part of the scoring of a load case.

The overall architecture of the Python program was informed through researching object-oriented coding practices and combining it into some of the structure that was already present in the original code. The program uses four main python scripts:

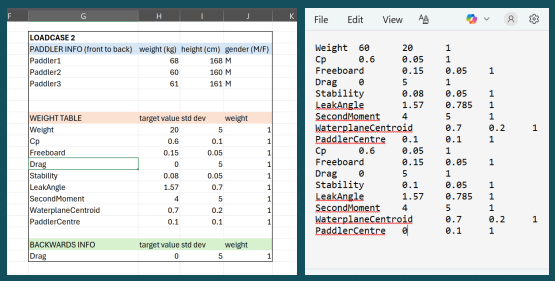
1. math_utils.py: This script has general math functions used to calculate areas and moments of sections defined by cubic splines.
2. canoe.py: This script is separated into two main classes, the Loadcase class and the Canoe class. The loadcase class serves as a storage for every loadcase and includes information on paddlers, target values, and backwards cases, as well as loadcase-related functions. The canoe class serves as another storage for canoe specific parameters (the code in main.py calls this canoe class once per canoe iteration). This canoe class contains most of the canoe-specific calculations like finding the water line and the keel line.
3. revised_possum_calc.py: This script helps optimize the search space of canoe parameters (as seen in the canoe tab of the excel below). It is run by main, in between bulk analysis iterations to narrow down the search space until it eventually converges.
4. main.py: This script imports the three scripts above; handles inputs, outputs, and the GUI; instantiates the classes and uses the functions in the other scripts as needed, and combines their functionalities to perform a bulk analysis over a defined search space of canoe parameters (as seen in the figure below) or an analysis of a single canoe.

The team coordinated work on this project through weekly discussions on the code progress, research findings, and ideas. Members worked on various functions based on their skill and interest. Version control was done through GitHub and integration testing was done together as a team. Claude Sonnet 4.5 [24] was used to speed up the runtime of the program through multiprocessing in main.py [25][26] and caching and vectorized calculations in canoe.py [25][27]. This posed very little risk to the accuracy of the program, since the code was

only edited for speed but the logic itself was not changed. All edited lines were also reviewed for accuracy prior to merging. Once the program was complete, the team also performed hand calculations to check the code, which was facilitated by the code's improved transparency of geometric values. Additionally, it was determined from research that security risks through this implementation of AI were minimal, as the information provided to Claude [24] was not deemed as sensitive, and the agent used has various features in place to protect user privacy and data security [28][29].

Impact & Results

Overall, the approach taken met the problem statement requirements. It had the new capability of processing inputs regarding all race types (including any number of paddlers and the backwards case, which may vary by year) and all paddler information (including gender, height, and weight which may also be variable by year) [Figure 1]. It now leverages this information to make fewer generalizations and assumptions, and thus produces more relevant and accurate results.



LOADCASE 2			
PADDLER INFO (front to back)	weight (kg)	height (cm)	gender (M/F)
Paddler1	68	168	M
Paddler2	60	160	M
Paddler3	61	161	M
WEIGHT TABLE	target value	std dev	weight
Weight	20	5	1
Cp	0.6	0.1	1
Freeboard	0.15	0.05	1
Drag	0	5	1
Stability	0.08	0.05	1
LeakAngle	1.57	0.7	1
SecondMoment	4	5	1
WaterplaneCentroid	0.7	0.2	1
PaddlerCentre	0.1	0.1	1
BACKWARDS INFO	target value	std dev	weight
Drag	0	5	1

```
Weight  60      20      1
Cp      0.6     0.05    1
Freeboard       0.15    0.05    1
Drag    0       5       1
Stability       0.08    0.05    1
LeakAngle       1.57    0.785   1
SecondMoment    4       5       1
WaterplaneCentroid      0.7     0.2     1
PaddlerCentre   0.1     0.1     1
Cp      0.6     0.05    1
Freeboard       0.15    0.05    1
Drag    0       5       1
Stability       0.1     0.05    1
LeakAngle       1.57    0.785   1
SecondMoment    4       5       1
WaterplaneCentroid      0.7     0.2     1
PaddlerCentre   0       0.1     1
```

Figure 1: New input Excel file loadcase (left) compared to original text file (right).

To quantify this change, last year's canoe, *C4NO3H*, was run through the new software and compared to the output from the old software, and the output for the new canoe. These results show a lower freeboard in the 4-paddler co-ed race, which went unaddressed as the hard-coded weights were lower than that of our paddlers. With this more comprehensive analysis in the design of *Astronautical*, we were able to ensure higher freeboard for all cases. Additionally, the backwards case implementation reveals that the difference in drag between the forwards and backwards case for 3 paddlers was significant, whereas this year the difference is much smaller. These two improvements alone make for a safer and more well-informed paddler experience. Finally, the clearer output of the results also allowed for easier interpretation of the results (i.e. which loadcase they pertain to), allowing for better informed changes in our iteration process that resulted in *Astronautical*. The notable improvements these software changes have made in our hull generated hulls are summarized in Table 1. These changes also introduced better maintainability and usability of the script, which make modifications and fixes in future years much easier now that it has been cleaned up and been made more modular. The program now also includes a simpler user interface and more user input flexibility via an excel file.

Table 1: Improvements to canoe hull design made possible by code improvements. Drag calculations assume a smooth hull surface with a sharp bow and stern, and 5 knots speed.

Characteristic	***C4NO3H* (2025) (old)**	***C4NO3H* (2025) (new)**	***Astronautical* (2026)**
Lowest Freeboard (cm)	14.8	12.2	**16.4**
Forwards Drag-Backwards Drag (lbs)	N/A	0.419	**0.050**

Future Improvements

In the future, the team is looking to continue to improve this program by improving its usability (including making the program easier to install and implementing a GUI), creating some comprehensive documentation, and optimizing the way the search space of canoe parameters is explored. Additionally, the team wants to make the thickness of the canoe a configurable parameter, add elements structural analysis that are currently only hand calculated, and improve the freeboard calculations to consider a dynamic freeboard rather than a static one (and suggest recommended paddler positions for different racing cases).